Dla chętnych: od InstaLogik do assembly AQA i LMC

For keen pupils: from InstaLogik to AQA assembly and LMC

Dodatkowa praktyka po opanowaniu materiałów konkursowych – ściągawka i przykłady.

Extra practice once the contest materials are mastered – a cheat sheet and examples.

L.O. – Cele lekcji / Learning objectives

- Rozumiem, że język z konkursu to uproszczony **assembly** – tak samo działają prawdziwe procesory.
  *I understand that the contest language is a simplified **assembly** language – real processors work the same way.*
- Potrafię zamienić proste polecenie InstaLogik na polecenie assembly AQA.
  I can turn a simple InstaLogik command into an AQA assembly command.
- Wiem, że **rejestry** to „pudełka" procesora, a **etykiety** zastępują numery linii.
  *I know that **registers** are the processor's "boxes" and **labels** replace line numbers.*

To materiał dodatkowy, nie przygotowanie do konkursu: składnia jest inna niż w InstaLogik, więc programów z zeszytów nie da się w nim sprawdzać. Zewnętrzny symulator (Peter Higginson, strona „AQA Assembly Language Simulator") działa w przeglądarce. Sposób wprowadzania i wypisywania danych w nim jest inny niż w konkursie – zajrzyj do jego pomocy; poniższe przykłady używają stałych liczb i wynik czytamy z rejestrów.

This is extra material, not contest preparation: the syntax differs from InstaLogik, so workbook programs cannot be checked in it. The external simulator (Peter Higginson's "AQA Assembly Language Simulator") runs in the browser. Input and output work differently from the contest, so see its help page; the examples below use fixed numbers and we read the result from the registers.

1. Słownik: InstaLogik ↔ AQA / 1. Dictionary: InstaLogik ↔ AQA

InstaLogik	assembly AQA	Uwaga / Note
`Ustaw A na 5`	`MOV R0, #5`	# przed liczbą *# before a number*
`Ustaw A na B`	`MOV R0, R1`	bez #, bo to rejestr *no #, it is a register*
`Zwiększ A o 3`	`ADD R0, R0, #3`	wynik, pierwszy składnik, drugi składnik *result, first value, second value*
`Zwiększ A o B`	`ADD R0, R0, R1`	
`Zmniejsz A o 3`	`SUB R0, R0, #3`	
`Skocz do 7`	`B etykieta`	etykieta: stoi w kodzie zamiast numeru linii *a label: replaces the line number*
`Jeżeli A = B skocz do…`	`CMP R0, R1` `BEQ etykieta`	najpierw porównaj, potem skocz *compare first, then branch*
`Jeżeli A ≠ B skocz do…`	`CMP R0, R1` `BNE etykieta`	
`Jeżeli A > B skocz do…`	`CMP R0, R1` `BGT etykieta`	
`Jeżeli A < B skocz do…`	`CMP R0, R1` `BLT etykieta`	
`Jeżeli A ≥ B skocz do…`	`CMP R0, R1` `BGT etykieta`	w AQA nie ma osobnego ≥ ani ≤: łączymy dwa skoki *AQA has no separate ≥ or ≤: combine two branches*

	`BEQ etykieta`	
`Skocz do końca`	`HALT`	zatrzymuje program *stops the program*
`pudełka A, B, C, D`	`rejestry R0, R1, R2, R3 …`	do R12 *up to R12*
`numer linii`	`etykieta`	

Ważna różnica: w InstaLogik w poleceniu „Jeżeli” podajemy oba miejsca skoku („…inaczej skocz do…”). W assembly AQA po niespełnionym warunku program po prostu idzie do **następnej linii**.

Key difference: *in InstaLogik the “Jeżeli” command names both jump targets (“…inaczej skocz do…”). In AQA assembly, when the condition is not met the program simply goes on to the **next line**.*

2. Przykłady / 2. Examples

Przykład 1: podwajanie (liczba 7) / Example 1: doubling (the number 7)

InstaLogik – wynik w pudełku A / result in box A

```
1. Ustaw A na 7
2. Zwiększ A o A
3. Skocz do końca
```

assembly AQA – wynik w R0 = 14 / result in R0 = 14

```
MOV R0, #7
ADD R0, R0, R0
HALT
```

Przykład 2: większa z dwóch liczb (3 i 5) / Example 2: the larger of two numbers (3 and 5)

InstaLogik – wynik w A / result in A

```
1. Ustaw A na 3
2. Ustaw B na 5
3. Jeżeli A > B skocz do końca inaczej skocz do następnej
4. Ustaw A na B
```

assembly AQA – wynik w R0 = 5 / result in R0 = 5

```
MOV R0, #3
MOV R1, #5
CMP R0, R1
BGT end
MOV R0, R1
end:
HALT
```

Przykład 3: mnożenie pętlą (3 × 4) / Example 3: multiplication with a loop (3 × 4)

assembly AQA – wynik w R0 = 12 / result in R0 = 12

```
MOV R0, #0
MOV R1, #3
MOV R2, #4
loop:
```

```
CMP R1, #0
BEQ done
ADD R0, R0, R2
SUB R1, R1, #1
B loop
done:
HALT
```

R1 zlicza powtórzenia, R2 to dodawana liczba, R0 zbiera wynik. Pętla kończy się, gdy R1 dojdzie do 0.

R1 counts the repeats, R2 is the number being added and R0 collects the result. The loop ends when R1 reaches 0.

Przykład 4: pamięć / Example 4: memory

assembly AQA – po wykonaniu R1 = 9 / after running R1 = 9

```
MOV R0, #9
STR R0, 100
LDR R1, 100
HALT
```

W InstaLogik mamy tylko cztery pudełka. Prawdziwy procesor ma też **pamięć**: STR zapisuje rejestr pod numerem adresu, LDR odczytuje.

*InstaLogik has only four boxes. A real processor also has **memory**: STR stores a register at an address number and LDR reads it back.*

3. Little Man Computer (LMC) / 3. Little Man Computer (LMC)

LMC to prosty model komputera do nauki w szkołach (np. strony peterhigginson.co.uk/lmc i 101computing.net/lmc). Ma jeden rejestr roboczy, **akumulator**, oraz „skrzynki” na liczby, a wejście i wyjście działają jak w konkursie: INP wczytuje liczbę, OUT ją wypisuje.

*LMC is a simple computer model for school teaching (e.g. peterhigginson.co.uk/lmc and 101computing.net/lmc). It has a single working register, the **accumulator**, and “mailboxes” for numbers, and input and output work as in the contest: INP reads a number and OUT prints it.*

InstaLogik	LMC	Uwaga / Note
`Wczytaj do A`	`INP` `STA a`	INP wczytuje do akumulatora, STA zapisuje do skrzynki a *INP reads into the accumulator, STA stores it in mailbox a*
`Wypisz pudełko A`	`LDA a` `OUT`	najpierw wczytaj skrzynkę do akumulatora *first load the mailbox into the accumulator*

`Zwiększ A o B`	`LDA a`
	`ADD b`
	`STA a`
		wszystkie działania idą przez akumulator
		all arithmetic goes through the accumulator

`Zmniejsz A o B`	`LDA a`
	`SUB b`
	`STA a`	

`Skocz do 7`	`BRA etykieta`	etykieta stoi na początku linii *the label sits at the start of the line*
`Jeżeli A = 0 skocz do…`	`LDA a` `BRZ etykieta`	BRZ: skok, gdy akumulator = 0 *BRZ: jump if the accumulator is 0*

`Jeżeli A ≥ B skocz do…`	`LDA a`
	`SUB b`
	`BRP etykieta`
		BRP: skok, gdy wynik ≥ 0
		BRP: jump if the result is ≥ 0

`Skocz do końca`	`HLT`	
`pudełka A, B…`	`skrzynki: a DAT, b`	DAT rezerwuje miejsce, można nadać wartość początkową

	`DAT 0`	*DAT reserves a place and can give a start value*

Inne porównania (np. A > B) wymagają połączenia kilku poleceń. W LMC nie ma też osobnego „Jeżeli”: najpierw liczymy w akumulatorze, potem skaczemy. W niektórych symulatorach liczby ujemne działają inaczej niż w konkursie, więc pilnuj, by wyniki były dodatnie.

Other comparisons (e.g. A > B) need several commands combined. LMC also has no separate “Jeżeli”: first calculate in the accumulator, then branch. In some simulators negative numbers behave differently from the contest, so keep results positive.

LMC – przykład 1: podwajanie / LMC – example 1: doubling

LMC – wpisz 7, wypisze 14 / enter 7, prints 14

```
INP
STA x
ADD x
OUT
HLT
x DAT
```

LMC – przykład 2: większa z dwóch liczb / LMC – example 2: the larger of two numbers

LMC – wpisz 3 i 5, wypisze 5 / enter 3 and 5, prints 5

```
INP
STA a
INP
STA b
LDA a
SUB b
BRP first
LDA b
OUT
HLT
first LDA a
OUT
HLT
a DAT
b DAT
```

LMC – przykład 3: mnożenie pętlą / LMC – example 3: multiplication with a loop

LMC – wpisz 3 i 4, wypisze 12 / enter 3 and 4, prints 12

```
INP
STA a
INP
STA b
loop LDA b
BRZ done
SUB one
STA b
LDA total
ADD a
STA total
BRA loop
done LDA total
```

```
OUT
HLT
a DAT
b DAT
total DAT 0
one DAT 1
```

4. Zadania dla chętnych / 4. Challenges

1. Przepisz na assembly AQA program z InstaLogik: „Ustaw A na 10, Zmniejsz A o 4”. Ile wynosi R0?
   Rewrite the InstaLogik program “Ustaw A na 10, Zmniejsz A o 4” in AQA assembly. What is R0?
2. Zmień przykład 2 tak, aby wynikiem była **mniejsza** z dwóch liczb.
   *Change example 2 so that the result is the **smaller** of the two numbers.*
3. Zmień przykład 3 tak, aby liczył 5 × 6. Które linie trzeba zmienić?
   Change example 3 to calculate 5 × 6. Which lines must change?
4. Napisz program, który dodaje liczby od 1 do 5 (wynik 15) pętlą z etykietą.
   Write a program that adds the numbers 1 to 5 (result 15) using a loop with a label.
5. W LMC napisz program, który wczytuje liczbę n i wypisuje sumę 1 + 2 + … + n.
   In LMC write a program that reads a number n and prints the sum 1 + 2 + … + n.

Odpowiedzi (dla nauczyciela) / Answers (for the teacher)

1 → R0 = 6

```
MOV R0, #10
SUB R0, R0, #4
HALT
```

2 → BLT zamiast BGT; dla 3 i 5 R0 = 3 / BLT instead of BGT; for 3 and 5, R0 = 3

```
MOV R0, #3
MOV R1, #5
CMP R0, R1
BLT end
MOV R0, R1
end:
HALT
```

Zadanie 3: zmień linie z liczbami MOV R1, #5 i MOV R2, #6 (wynik 30).

Challenge 3: change the lines MOV R1, #5 and MOV R2, #6 (result 30).

4 → R0 = 15

```
MOV R0, #0
MOV R1, #1
loop:
CMP R1, #5
BGT done
ADD R0, R0, R1
ADD R1, R1, #1
B loop
done:
HALT
```

5 (LMC) → dla n = 4 wypisze 10 / for n = 4 prints 10

```
INP
STA n
LDA zero
STA total
loop LDA n
BRZ done
LDA total
ADD n
STA total
LDA n
SUB one
STA n
BRA loop
done LDA total
OUT
HLT
n DAT
total DAT 0
zero DAT 0
one DAT 1
```